

How KOHRA Answers a Question

Official Technical Specification Document (compiled from docs/prompts.md)

This document walks through every prompt, rule, and workflow the KOHLER Enterprise Intelligence Agent executes when a user submits a question. It covers the one LLM call the system makes, the seven deterministic stages around it, and how the knowledge base was built.

The one LLM call

KOHRA uses a single model for all answer generation: Groq's **llama-3.3-70b-versatile** at **temperature=0.1** with **response_format={ type: "json_object" }**. Every other pipeline stage — routing, retrieval, access control, conflict detection, safety gating, formatting — is deterministic code. No LLM calls happen outside of answer generation.

When the Groq API key is missing or the call fails, a deterministic synthesizer (**backend/src/services/answer.ts:21-202**) produces pre-authored answers by matching the user's query against regex patterns. This keeps the demo functional offline.

The system prompt

Set once per conversation, applied to every answer generation call:

```
You are the KOHLER Enterprise Intelligence Agent, an authoritative policy assistant.  
Answer the user's question using ONLY the provided CONTEXT DOCUMENTS.
```

RULES:

1. Ground every statement in the provided documents. Never speculate or hallucinate.
2. If a policy has superseded an older version (e.g. v3.0 replacing v2.0), explicitly state the newer active terms and note the update.
3. If documents do not contain the answer, say "I don't have sufficient information about this in the knowledge base."
4. Extract precise citations matching the documents you used.
5. You MUST return ONLY a valid JSON object matching this exact schema without any markdown wrapping:

```
{  
  "answer": "string (clear, professional markdown text)",  
  "confidence": 0.0 to 1.0,  
  "grounded": true,  
  "citations": [  
    {  
      "documentTitle": "string",  
      "section": "string",  
      "version": "string",  
      "snippet": "exact brief text snippet from doc",  
      "filePath": "optional relative path to markdown file"  
    }  
  ]  
}
```

The five rules are the behavioral spine: grounded-only answers, supersession disclosure when newer policy versions exist, graceful refusal with department redirection when context is missing, mandatory citation extraction, and strict JSON-only output with no markdown wrapping or prose outside the JSON structure.

The user prompt

Assembled fresh per query, injected with retrieved chunks, conflict warnings, conversation history, and the caller's RBAC role:

CONTEXT DOCUMENTS:

```
[DOC 1] Title: "{title}" | Section: "{section}" | Version: "{version}" | Domain: "{domain}" | File: "{filePath}"  
{chunk content}  
---  
[DOC 2] Title: "{title}" | Section: "{section}" | Version: "{version}" | Domain: "{domain}" | File: "{filePath}"  
{chunk content}
```

```

KNOWN POLICY CONFLICTS / SUPERSESSIONS:
- [SUPERSEDED] Policy "{title}" v{version} supersedes version {supersedes} (effective from {validFrom}).

CONVERSATION HISTORY:
User: {previous user message}
Assistant: {previous assistant response}
(or "None (new session)")

USER ROLE: {rbacRole}

USER QUESTION:
{query}

RESPONSE (JSON only):

```

Chunks are numbered [DOC 1], [DOC 2], etc. Each chunk's header includes title, section, version, domain, and file path so the LLM can cite precisely. Conflict warnings only appear when the conflict detector found issues. The last three conversation turns are included for multi-turn context. The user's RBAC role is stated explicitly — RBAC is enforced pre-retrieval, but the LLM sees the role to understand access context.

What happens before the LLM sees anything

Seven stages execute between the user pressing Send and the model being called. Several of these stages can short-circuit the pipeline entirely.

1. Query validation and session setup

The query is checked for non-empty content. A session is created or restored (30-minute TTL, in-memory **Map**). The last three conversation turns are loaded for multi-turn context. The **QUERY_RECEIVED** audit event is logged.

2. Domain classification

The router (**backend/src/services/router.ts**) scores the query against five domain anchors — HR, Finance, Support, Privacy, Legal — using keyword overlap and example-query similarity. No embedding model, no LLM call. Just string tokenization and arithmetic.

Each anchor has a keyword list and a set of example queries. The scoring formula:

```

baseScore = matchCount / (queryTokens.length + 1)
combinedScore = min(1.0, baseScore * 0.6 + queryBonus * 0.4 + (matchCount > 2 ? 0.3 : 0))

```

Hardcoded phrase bonuses give extra weight to known high-signal patterns: "hr policies" (+5 to HR), "travel per diem" (+4 to Finance), "warranty" (+4 to Support), "share customer data" (+4 to Privacy and Legal simultaneously).

Three outcomes:

- **Score below 0.2:** The query is out of scope. The user gets: "Your question appears to be outside Kohler's policy knowledge base (HR, Finance, Support, Privacy, Legal). Would you like to connect with General IT or Human Resources?" Pipeline stops. No retrieval, no LLM call.
- **Score 0.2-0.45:** Ambiguous. The user gets: "Did you mean to ask about {DOMAIN} or another Kohler policy area?" Pipeline stops.
- **Score 0.45 or above:** The domain is set. If two or more domains score above 0.45, the system retrieves from all of them — this is how cross-domain questions like "Can we share customer data with a cloud vendor?" pull from both Privacy and Legal.

3. Hybrid retrieval

The retriever (**backend/src/services/retriever.ts**) runs two parallel searches over the domain-filtered chunks:

BM25 keyword search (k1=1.5, b=0.75) — a lazy-singleton inverted index rebuilt on first use. Good at matching exact terms: policy version numbers, dollar amounts, section headings.

Semantic scoring — not a vector database. Instead, the system expands the query into keyword variants and computes a combined relevance score using keyword overlap, title similarity, and section affinity. Hardcoded boosts handle known high-value

patterns: "travel per diem" gets +0.8 on title match and +0.6 on domestic/international sections; "warranty" gets +0.9 on title match; "executive entertainment" gets +0.9 on title match.

Results from both methods are merged using Reciprocal Rank Fusion: $RRF(d) = \frac{1}{\frac{1}{60} + \text{rank}_m(d)}$ across both rank lists. The top five chunks survive.

4. RBAC containment gate

After retrieval but before the LLM sees any documents, the RBAC gate (**backend/src/services/rbac.ts**) filters chunks by role membership. The rule is simple: **chunk.rbacTiers.includes(userRole)**. If a chunk's **rbacTiers** array does not contain the user's role, the chunk is dropped.

Six roles exist: **EMPLOYEE, MANAGER, HR, FINANCE, LEGAL, ADMIN**. An **EMPLOYEE** asking about supplier code of conduct gets all five retrieved chunks dropped — the Legal repository's supplier files require **LEGAL** or **ADMIN** tier. When every chunk is dropped, the pipeline returns an access-denied message without calling the LLM:

> "Access Restricted: This policy information requires elevated privileges ({droppedReason}). You are currently operating with the {rbacRole} role."

Every RBAC decision is audit-logged with **droppedCount** and per-document reasons.

5. Conflict and supersession detection

The conflict detector (**backend/src/services/conflictDetector.ts**) runs three checks on chunk metadata — no LLM involvement:

1. **Supersession:** If **chunk.supersedes** is set, a warning is emitted. This catches version chains like the PTO policy (v1.0 superseded by v2.0, superseded by v3.0).
2. **Date expiry:** If **chunk.validUntil** is before today's date, the chunk is flagged as expired.
3. **Explicit conflicts:** If **chunk.conflictsWith** lists other policies, an informational warning notes the relationship.

Warnings are deduplicated by message text and injected into the LLM's user prompt under **KNOWN POLICY CONFLICTS / SUPERSESSIONS**, so the model can address versioning explicitly in its answer.

6. HITL safety gate

The Human-in-the-Loop gate (**backend/src/services/hitlGate.ts**) scans the query for risk-triggering keywords and assigns one of four levels:

CRITICAL — blocks the response, escalates to HR. Triggers: "executive compensation", "c-suite bonus", "salary band", "confidential severance formula".

HIGH — blocks the response, escalates to LEGAL. Triggers: "breach notification" + "authority", "vendor data sharing" + "exception", "override encryption".

MEDIUM — does not block, but appends an advisory. Triggers: "policy exception", "budget override", "waive approval threshold". The user sees a recommendation to file a formal approval ticket with their manager.

LOW — the default for standard policy inquiries. No escalation.

7. Answer generation

Only after all six preceding stages pass does the system assemble the user prompt (Section 1.2) and call the Groq API. The response is parsed as JSON. If parsing fails, the system retries once with a correction prompt. If the second attempt also fails, the offline deterministic synthesizer takes over.

Post-generation, citations are enriched: any missing **filePath** fields are filled by matching **documentTitle** against the original retrieved chunks.

The offline deterministic synthesizer

When no Groq API key is configured, or the API returns invalid JSON after retry, **backend/src/services/answer.ts** falls through to a deterministic synthesizer that pattern-matches the query against seven known scenarios:

Pattern	What it answers	Confidence
"travel per diem" or "per diem" + "domestic"	\$75/day breakdown, pre-approval rules, receipt thresholds	0.95
"share customer data" or "cloud vendor" + "data"	4-point response: DPA, MSA, Cross-Border SCCs, Dual Sign-Off	0.92
"executive entertainment" or "entertainment budget"	VP \$50K, SVP \$120K, per-event \$250/person, golf \$15K	0.96
"leave allowance" or "pto allowance" or "vacation" + "days"	20 days PTO v3.0, 25 days at 5+ years, 8-day carryover	0.94
"all hr policies" or "list" + "hr"	4-policy catalog: PTO, Benefits, Remote Work, Severance	0.98
"warranty" or "return" + "india"	Warranty coverage, return policy, replacement service	0.95
"employee privacy" or "collect" + "employee"	3-category data collection: PII, employment, security/IT	0.95
No match	First retrieved chunk truncated to 300 characters	0.88

If no chunks were retrieved at all, the response is: "I don't have sufficient information in the accessible Kohler knowledge base to answer your question. Please contact HR, IT Support, or Corporate Compliance for assistance."

Output formatting

After the LLM returns its JSON answer, the formatter (**backend/src/services/formatter.ts**) can render it in five formats:

- Prose** — answer text plus a markdown citation list. The default.
- JSON** — the full structured response: requestId, query, answer, domain, confidence, grounded flag, citations, timestamp.
- XML** — a root element with all fields XML-escaped.
- Excel** — a SheetJS workbook with three sheets: Summary (query + answer + metadata), Source Citations (title, section, version, snippet, file path for each citation), Governance Metadata (RBAC role, domain, HITL status, request ID, timestamp). Saved to **/exports/** and returned with a download URL.
- Email** — rendered through a Handlebars template. Recipient is routed by role: FINANCE goes to **finance-approvals@kohler.com**, HR to **people-support@kohler.com**, LEGAL to **compliance-legal@kohler.com**, MANAGER to **leadership-desk@kohler.com**, ADMIN to **enterprise-admin@kohler.com**, EMPLOYEE to **employee-inquiry@kohler.com**. All emails CC **compliance-team@kohler.com**. Subject line uses the first seven words of the query.

The audit trail

Every request generates a full audit log (**backend/src/services/auditLog.ts**) stored in memory, indexed by **requestId** and **sessionId**. Eight events are tracked:

Event	When	Severity
"QUERY_RECEIVED"	User submits query	INFO
"ROUTER_CLASSIFIED"	Domain classification complete	INFO
"RETRIEVAL_EXECUTED"	Chunks retrieved	INFO
"RBAC_FILTERED"	RBAC gate applied	WARNING if chunks dropped
"CONFLICT_DETECTED"	Conflicts found	WARNING

"HITL_REQUIRED"	High-risk query escalated	WARNING
"ANSWER_GENERATED"	LLM response received	INFO
"RESPONSE_SENT"	Final answer formatted and sent	INFO

Each event records timing: **routeMs**, **retrieveMs**, **rbacMs**, **conflictMs**, **hitlMs**, **answerMs**, **formatMs**, **totalMs**.

How the knowledge base was built

Document ingestion

The ingestion pipeline (**backend/src/data/loader.ts**) reads Markdown files from **backend/policies/**, organized into five domain directories: **finance/**, **hr/**, **support/**, **privacy/**, **legal/**. A sixth directory, **0_official_policies/**, holds source PDFs and original Markdown used for extraction but is skipped during ingestion.

Each **.md** file has YAML frontmatter (parsed via **gray-matter**) containing: title, domain, version, validFrom, validUntil, supersedes, keywords, rbacTiers, conflictsWith, and filePath.

Content is chunked by **##** markdown headings. Sections exceeding 380 words (~500 tokens) are split with 40-word (~50 token) overlap. Each chunk gets a unique ID: **{domain}_{fileSlug}_{3-digit-index}**.

Official policy files

Nine official policy files were authored and added to the knowledge base during development:

Privacy (3 files, 42 chunks):

- font-mono: official-global-privacy-notice.md — 14-section global privacy notice covering data collection, use, sharing, retention, security, cross-border transfers, GDPR/CCPA rights, cookies, children's privacy, and contact details
- font-mono: official-employee-privacy-notice.md — 11-section employee privacy notice covering employment data collection, background checks, payroll, health information, monitoring, and international transfers
- font-mono: official-india-website-privacy-policy.md — 14-section India website privacy policy covering data collection, cookies, user rights, data retention, security measures, and grievance redressal

Support (3 files, 51 chunks):

- font-mono: official-global-terms-and-conditions.md — 18-section global terms covering product information, pricing, warranty, limitation of liability, dispute resolution, and force majeure
- font-mono: official-india-terms-explained.md — 13-section India terms covering pricing, GST, delivery, installation, warranty claims, return policy, and escalation paths
- font-mono: official-india-d2c-terms-of-service.md — 17-section India D2C terms covering account registration, pricing, payment methods, shipping, returns, warranty, and governing law

Legal (3 files, 47 chunks):

- font-mono: official-supplier-code-of-conduct.md — 23-section supplier code covering labor practices, health and safety, environmental compliance, anti-corruption, and monitoring
- font-mono: official-supplier-sustainability-policy.md — 13-section sustainability policy covering climate, water stewardship, circular economy, and Scope 3 reporting
- font-mono: official-global-impact-report.md — 8-section impact report covering climate action, water stewardship, equity and inclusion, and 2030 targets

All files use **rbacTiers**: **[EMPLOYEE, MANAGER, HR, FINANCE, LEGAL, ADMIN]** for open access. Total knowledge base: 206 chunks across 30 policy files, with 140 chunks from official documents.

Index build

After ingestion, **npm run rebuild-index** regenerates the chunk cache. The BM25 inverted index is rebuilt lazily on first retrieval request. No external vector database is used — semantic scoring runs entirely in-memory.

The scenario benchmark suite

Eight predefined scenarios (`backend/src/routes/scenarios.ts`) validate the pipeline end-to-end. Each scenario defines a query, expected domain, user role, and pass/fail assertions:

#	Name	What it tests	Key assertion
1	single-domain-grounding	Finance answer accuracy	Answer contains "\$75", at least 1 citation
2	cross-domain-synthesis	Privacy + Legal overlap	Answer contains "DPA" and "indemnification", 2+ citations
3	rbac-containment	Role-based access control	EMPLOYEE gets "Access Restricted"; FINANCE gets full answer
4	policy-conflict-detection	Version supersession	Answer contains "20 days", supersession warning emitted
5	multi-format-output	All 5 renderers	JSON, XML, XLSX, email, and prose all parse/validate
6	customer-support-terms	Support domain accuracy	Answer contains "warranty", at least 1 citation
7	rbac-supplier-containment	Legal RBAC enforcement	EMPLOYEE: 5 docs dropped; LEGAL: full answer
8	employee-privacy-notice	Privacy domain accuracy	Answer contains "data", at least 1 citation

Session management

Sessions (`backend/src/services/sessionStore.ts`) are in-memory only — no persistence to disk. A **Map** stores message turns with timestamps. TTL is 30 minutes from last access, with auto-eviction on read. Sessions are auto-created if missing or expired. The last three turns (six messages) are injected into the LLM context for multi-turn continuity.

Email template

The Handlebars template (`backend/src/templates/email.hbs`) renders policy answers as formal internal correspondence:

```
To: {recipientEmail}
Subject: Re: Kohler Policy Inquiry: {first 7 words of query}
Cc: compliance-team@kohler.com, policy-intelligence@kohler.com

Dear Kohler Team Member,

In response to your policy inquiry ("{query}"), here is the authoritative guidance
retrieved from Kohler Enterprise Knowledge Base:

{answer}

VERIFIED SOURCE CITATIONS:
{citations listing}

GOVERNANCE & AUDIT:
- RBAC Role Evaluated: {rbacRole}
- Domain Classification: {domain}
```

Development and Workflow Operations

The system architecture and codebase were designed and maintained using specialized agent workflows, skill protocols, and automated quality assurance suites.

1. Agent Protocol & System Architecture (`GEMINI.md`)

Development operations follow a strict 3-tier protocol:

- **TIER 0 (Universal Rules):** Mandatory Clean Code rules, Socratic Gate protocol, dependency checks via font-mono: CODEBASE.md, and language matching.
- **TIER 1 (Code & Execution Rules):** Intelligent agent routing based on task domain:
 - font-mono: @backend-specialist: Express API, RRF retrieval, RBAC filters, conflict detection, and dynamic serialization.
 - font-mono: @frontend-specialist: React 19, Vite, Kohler Foundations design tokens, state management.
 - font-mono: @security-auditor: Vulnerability scanning, RBAC containment auditing, secret protection.
 - font-mono: @project-planner / font-mono: @orchestrator: Multi-agent task planning and execution.
- **TIER 2 (Design System Governance):** Strict adherence to the Kohler Foundations Design System (Space Grotesk headings, Inter body typography, JetBrains Mono code/telemetry, Brand Vermilion font-mono: #ea3829, light heading scale font-mono: font-weight: 300, and Zero Purple rule).

2. Socratic Gate Protocol

Before executing any complex code edits or architectural modifications, every task passes through the Socratic Gate:

1. **Never Assume:** If any requirement or edge case is ambiguous, clarify before tool execution.
2. **Trade-off Evaluation:** Evaluate trade-offs (e.g., offline deterministic fallback vs Groq API JSON parsing).
3. **Plan Validation:** Confirm implementation steps prior to multi-file edits.

3. Developer Slash Command Workflows

The codebase supports 11 standardized operational workflows executed during development:

Slash Command	Workflow Purpose	Operational Output
"/orchestrate"	Master Multi-Agent Orchestration	Coordinates backend, frontend, security, and design specialists for complex builds.
"/plan"	4-Phase Task Planning	Produces structured "{task-slug}.md" documents covering Analysis, Planning, Solutioning, and Implementation.
"/brainstorm"	Socratic Requirement Discovery	Asks strategic discovery and trade-off questions to clarify ambiguous requirements.
"/create"	Full-Stack Application Builder	Builds full-stack features from natural language specs.
"/debug"	4-Phase Systematic Debugging	Executes root-cause investigation, evidence collection, traceback verification, and regression fixes.
"/test"	Test Execution & Verification	Runs the 8-scenario benchmark suite ("test-scenarios.ts"), unit tests, and Playwright E2E suites.
"/deploy"	Safe Production Deployment	Performs pre-flight security scans, bundle size profiling, and zero-secrets verification.

"/ui-ux-pro-max"	Design System Audit & Generation	Enforces Kohler Foundations Design System guidelines, light typography scales, and accessibility rules.
"/enhance"	Performance Optimization	Profiles Core Web Vitals, eliminates render waterfalls, and optimizes bundle chunking.
"/preview"	Visual UI Preview & Inspection	Renders interactive walkthrough artifacts and layout previews for design review.
"/status"	Workspace Health & Audit	Checks git status, policy ingestion counts (30 policies / 206 chunks), and benchmark pass rates.

4. Automated Verification & Quality Audit Scripts

Twelve specialized automation scripts (**.agent/scripts/** and **.agent/skills//scripts/**) audit code quality, security, performance, and UX compliance:

Script Path	Primary Function
"checklist.py"	Priority-ordered project quality audit (Security → Lint → Schema → Tests → UX → SEO).
"security_scan.py"	Vulnerability scanner for OWASP 2025 guidelines and secret leaks.
"dependency_analyzer.py"	Supply chain dependency vulnerability & license analyzer.
"lint_runner.py"	Code formatting and TypeScript static type compliance runner.
"test_runner.py"	Unit and integration test execution runner.
"schema_validator.py"	Data structure and API schema integrity validator.
"ux_audit.py"	Frontend UX guideline and design system compliance auditor.
"accessibility_checker.py"	WCAG accessibility and contrast ratio checker.
"seo_checker.py"	Semantic HTML and SEO fundamental validator.
"bundle_analyzer.py"	Production Javascript/CSS bundle size analyzer.
"lighthouse_audit.py"	Automated Lighthouse performance profiler.
"playwright_runner.py"	End-to-end browser user flow verification runner.